

Phishing Email Detection Using NLP & Machine Learning

A leakage-aware, recall-first comparison of four tuned classifiers on 82,486 real emails

The problem

Phishing is the most common way attacks enter an inbox, and nobody can review mail manually at scale. Given only an email's text, can a model catch the scams — and can we tell how much of its score is real signal versus dataset artefact?

82,486

emails in the raw corpus (52/48 balance)

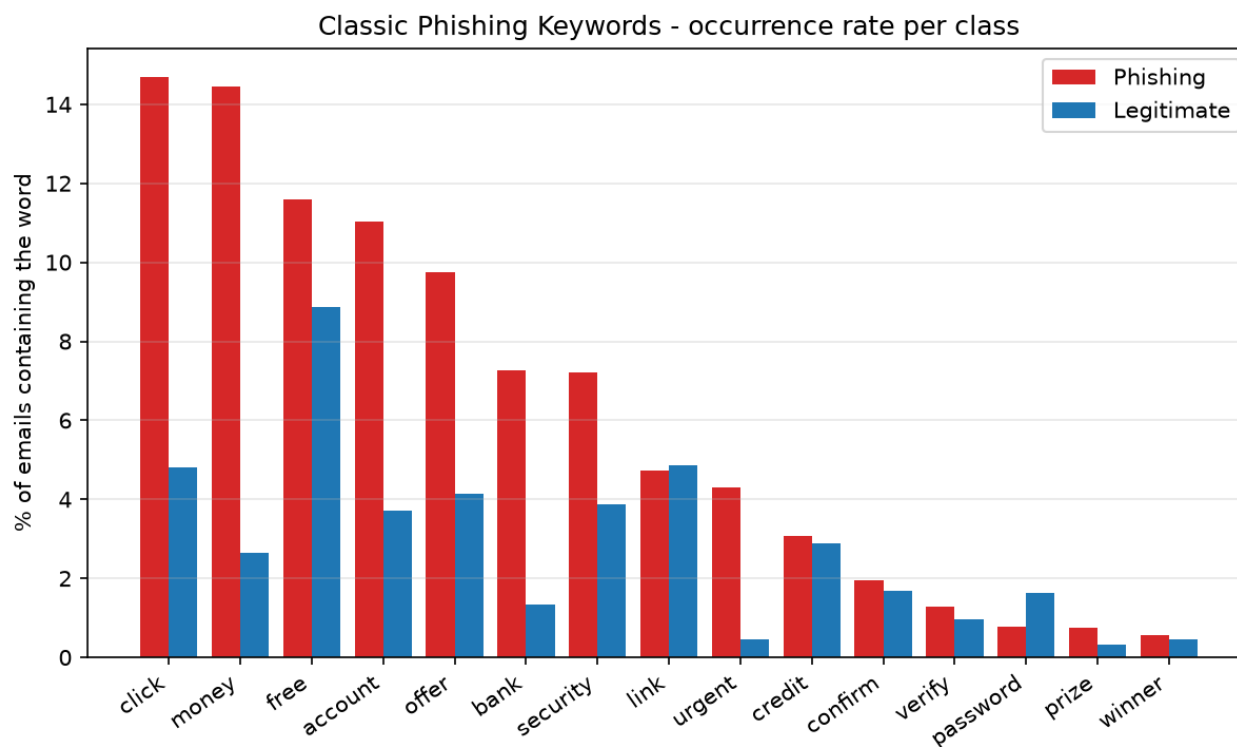
4

models, tuned & compared under identical conditions

98.5%

of phishing caught by the best model (recall)

Dataset: 82k real emails, measured before trusted



15 classic scam words, fixed before looking at the data — all more frequent in phishing ('click': 14.7% vs 4.8%)

1

Verified, not assumed

text arrives pre-normalised: 0.0% uppercase, 0.0% HTML — but 47.2% still carry URL tokens, so the pipeline still earns its place

2

Watermark measured & removed

'enron' in 18.2% of legitimate vs 0.0% of phishing → made a stop word

3

Outliers truncated first

longest email: 4,279,526 chars → capped at 20,000 (255 emails, 0.31%) before de-duplication

4

Clean corpus

409 duplicates removed, 0 label conflicts → 82,075 emails (52.2% phishing)

Methodology: split first, measure every choice

1. Clean

truncate → dedupe → conflict
check → normalise text ('enron'
stop word)

2. Split

stratified 80/20, seed 42 →
65,660 train / 16,415 test,
52.2% phishing both sides

3. Vectorize

fitted on training text only —
5,000 terms, no test-set
statistics leak in

4. Tune & compare

grid search where cheap; same
features, test set and metrics
for all four models

Representation decided by a pre-declared protocol

Representation	CV Accuracy	CV F1 (phishing)	Decision
CountVectorizer (5,000)	0.9795 ± 0.0014	0.9805 ± 0.0013	
TF-IDF (5,000)	0.9792 ± 0.0010	0.9801 ± 0.0010	SELECTED

Gap of 0.0003 — inside the 0.001 tie threshold declared before running. The tie rule picks TF-IDF (down-weights universal words; suggested by the brief). Same protocol chose CountVectorizer decisively in Project 1 — the default is not always the measured winner.

Four models, tuned where it pays

Same cleaned data • same 5,000 TF-IDF features • same stratified split • same metrics (phishing class) • seed 42 everywhere



Logistic Regression

C grid-searched: 0.1 / 1 / 10 → 10

CV F1 0.9801 → 0.9827 after tuning



Multinomial Naive Bayes

alpha grid-searched: 0.1 / 0.5 / 1.0 → 0.1

beats the 1.0 default (CV F1 0.9605)



Random Forest

100 trees, fixed

forests are insensitive to more trees —
budget went to grids that move the needle



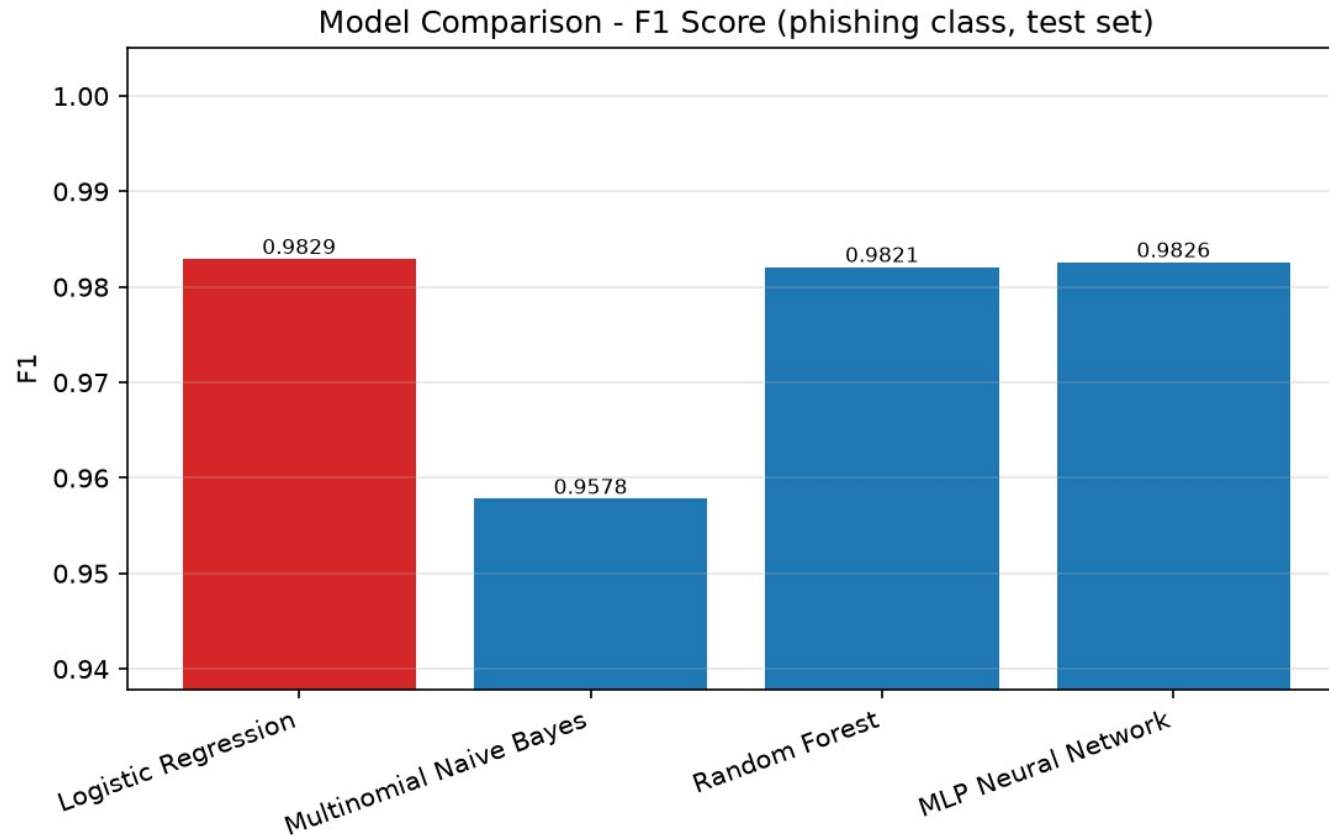
MLP Neural Network

100 hidden units • Adam • early stopping

self-regularises by construction

T grid-search tuned (3-fold CV, training set only) **F** fixed with documented rationale — grids, candidates and winners saved to hyperparameter_tuning.csv

Results: linear wins, recall decides



Confusion Matrix - Logistic Regression

True label	Predicted label	
	Legitimate (0)	Phishing (1)
Legitimate (0)	7,681 (46.8%)	165 (1.0%)
Phishing (1)	128 (0.8%)	8,441 (51.4%)

Best model: Logistic Regression (C=10)

98.22% accuracy • F1 0.9829 • recall 0.9851

293 errors / 16,415 test emails — 165 false alarms, 128 missed phish (≈15 per 1,000)

Top three within 0.0008 F1 • NB misses 498 phish (4× the winner) • 0.42 s vs MLP's 46.6 s training

Misses skew long: missed phishing averages 180 words vs 104 for phishing overall — short punchy scams are caught almost perfectly.

Conclusion

Measured, end to end

82,486 raw → 82,075 clean → tuned, saved model — one command, seed-reproducible

Recall-first winner

Logistic Regression (C=10): 98.22% acc · F1 0.9829 · recall 0.9851 · 128 misses in 8,569

Tuning earned its keep

C=10 lifted CV F1 0.9801 → 0.9827; alpha=0.1 beat the NB default

The honest finding

top features include archive names — josemonkeyorg, spamassassin tokens, Enron first names — partly a phishing detector, partly an archive detector

Future work

- Cross-archive evaluation
- Real URL & header features
- Calibrated probabilities
- Small transformer baseline
- Flask / Streamlit demo

Thank you — questions welcome. · Everything reproduces with `python main.py` (seed 42).